PROJECT REPORT

Typeahead: A Prefix Search Engine

Distributed Caching, Trie-Based Serving, and Write-Back Batching

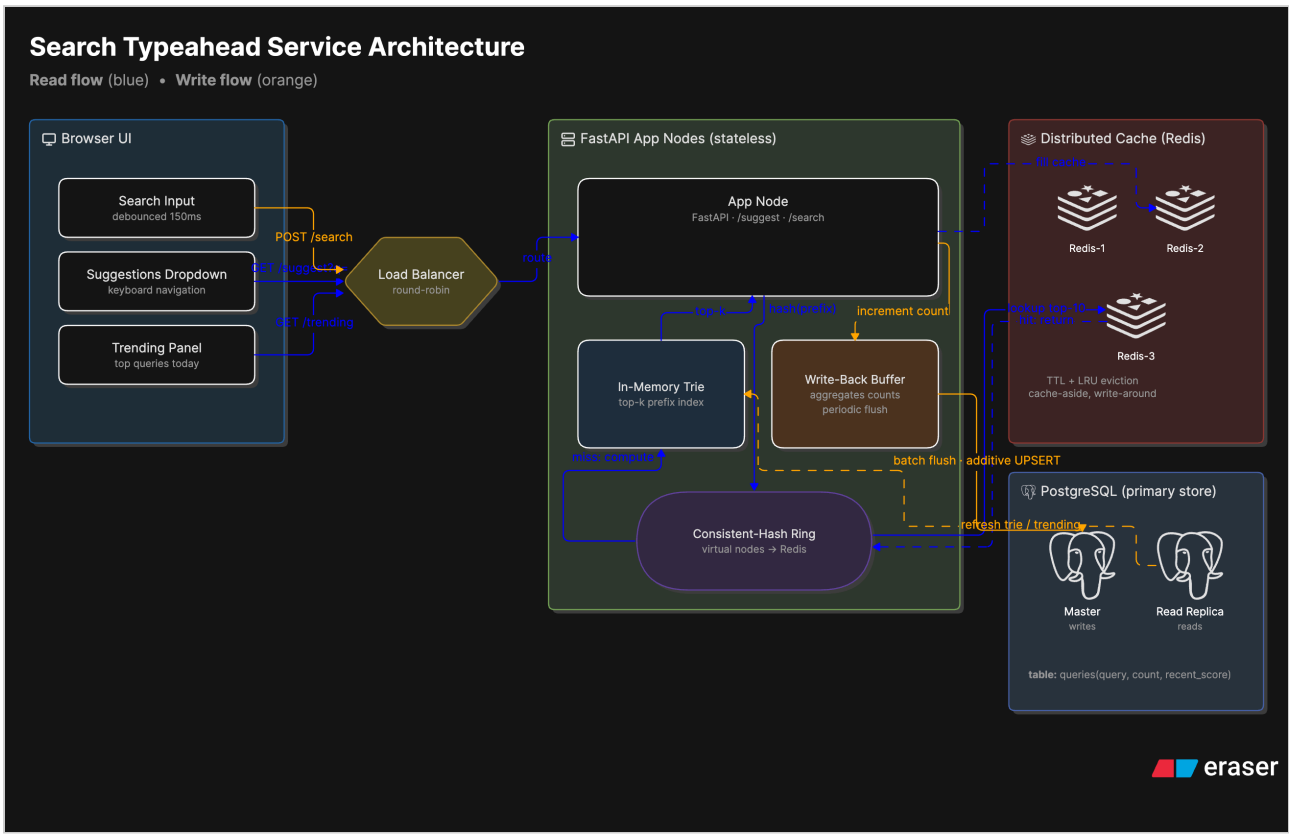
A read-optimized search-autocomplete service: suggestions are served from a distributed cache (routed by consistent hashing) backed by an in-memory trie, while submitted searches are persisted via write-back batching into PostgreSQL. Implemented on Next.js 15 (App Router and Route Handlers) with TypeScript.

System	Search-autocomplete / typeahead service
Stack	Next.js · TypeScript · PostgreSQL · 3× Redis · React
Consistency model	Eventual (PACELC: PA/EL)
Date	June 2026

CONTENTS

01 Architecture

The system has two workloads on one dataset that pull in opposite directions. Every keystroke is a **read** (a suggestion lookup); every submitted query is a **write** (a count update). Reads outnumber writes by roughly 5–10×. Two facts drive the whole design: reads are on the hot path and must stay fast, and suggestion data is *approximate popularity* — slight staleness is harmless.



System architecture — distributed cache, trie serving, write-back batching to Postgres.

The two paths

Read path (cache-aside). `GET /api/suggest` hashes the prefix on a consistent-hash ring to pick one of three Redis nodes. A **HIT** returns the cached top-10 immediately. A **MISS** is answered by the in-memory trie — never by Postgres — and the result is written back into Redis under a jittered TTL so the next identical request is a HIT.

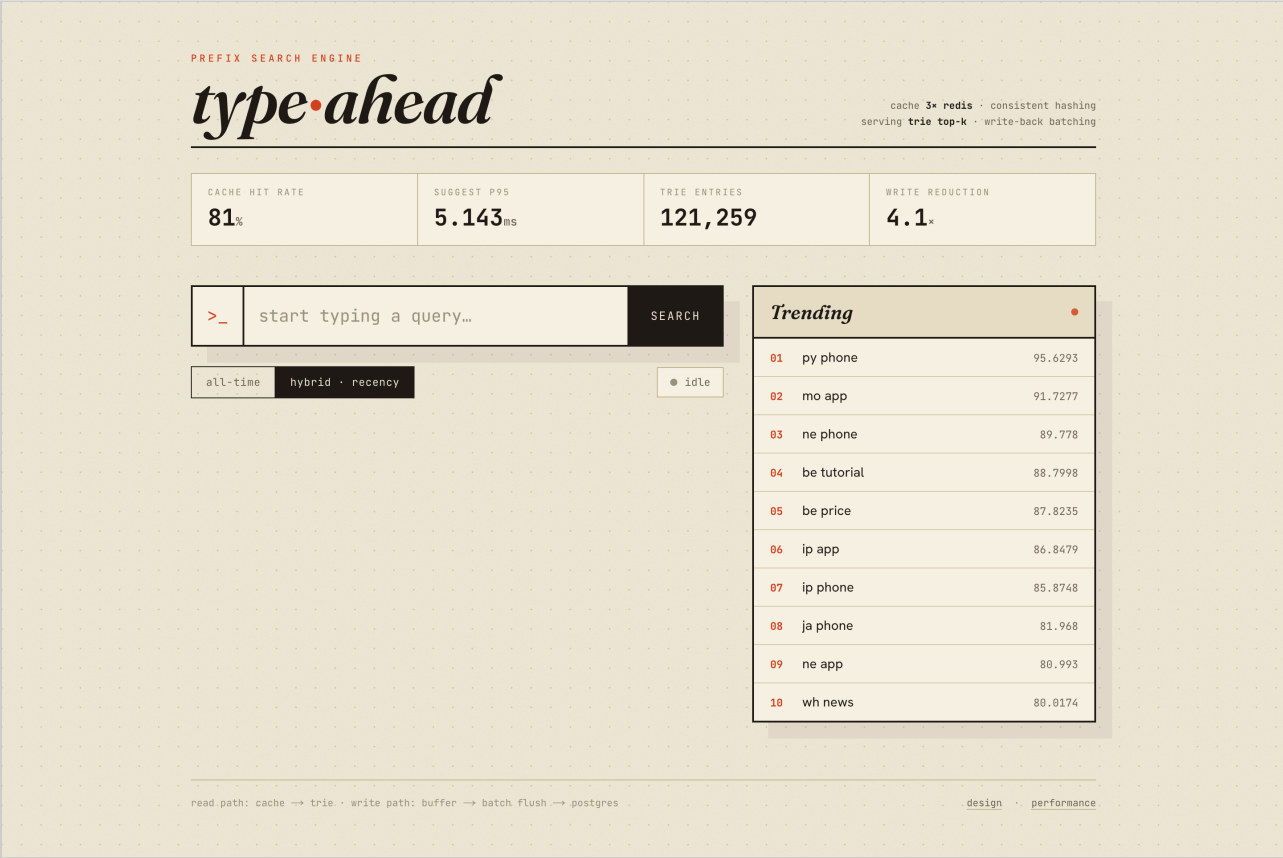
Write path (write-back batching). `POST /api/search` increments an in-memory map and returns an acknowledgement instantly. A buffer aggregates duplicate queries and flushes on size (500 entries) or interval (1s) as one additive `UPSERT`, then mirrors the same deltas into the trie so reads reflect new activity without waiting for the next rebuild.

```
READ  (every keystroke)                WRITE (every submitted search)
browser → GET /api/suggest             browser → POST /api/search
   → Route Handler                        → Route Handler
   → Redis (1 of 3, hash ring)            → in-memory WriteBuffer (a Map)
   → HIT: return cached top-10            → flush on 500 entries OR 1s
```

```
    → MISS: in-memory Trie → top-10      → Postgres (one additive UPSERT)
    → backfill cache, return             → mirror deltas into the Trie
```

Components

Component	Role
Next.js Route Handlers	Stateless API tier (suggest, search, trending, metrics, cache introspection).
In-memory Trie	O(prefix length) prefix top-k; serves every cache miss. Same structure serves both ranking modes.
3× Redis (consistent hashing)	Global distributed cache of computed top-10 lists; sharded for fault tolerance.
Write buffer	Aggregates submitted searches in memory; flushes in batches (write-back).
PostgreSQL	Durable source of truth for counts; B-tree index for prefix range scans.
Runtime singleton	Builds the trie + buffer + Redis pool once and caches them on `globalThis` (the Next.js equivalent of Express's single startup hook).



Running application — live gauges (hit rate, p95, trie size, write reduction), suggestions, and trending.

02 Dataset — source & loading

Source. The AOL 2006 query log — real user searches. Per-query counts are derived by aggregation (`COUNT` per normalized query). The raw log is ~35.4M rows that aggregate to ~4.1M distinct queries; we load the **top 1,000,000 by count** (the full set would make the in-memory trie unnecessarily large).

For a zero-download quick start, a `--synthetic N` generator produces Zipf-distributed queries with realistic popularity skew.

Loading the real dataset

```
curl -L -o files/aol.zip
"https://archive.org/download/AOL_search_data_leak_2006/AOL_search_data_leak_2006.zip"
unzip -o -j files/aol.zip "AOL-user-ct-collection/*.txt.gz" -d files/aol_data

# aggregate raw logs to a counts file
npm run load -- --dir files/aol_data --min-count 2 --out files/aol_agg.tsv

# ingest the top 1M into Postgres
npm run load -- --agg-file files/aol_agg.tsv --top 1000000 --min-count 3
```

Quick start (synthetic, no download)

```
docker compose up -d                  # Postgres + 3 Redis
cp .env.example .env
npm install
npm run load -- --synthetic 120000    # generate + ingest 120k queries
npm run build && npm start            # serve on http://127.0.0.1:8000
```

> The ingestion script (`scripts/loadDataset.ts`) streams gzipped log files line-by-line, normalizes and counts queries in a map, filters by a minimum count, and bulk-inserts in 1,000-row chunks — so even the full log fits in bounded memory.

03 API documentation

All endpoints are Next.js Route Handlers under `app/api/`, running on the Node.js runtime. Base URL in development: `http://127.0.0.1:8000`.

Method	Endpoint	Description
GET	/api/suggest?q=<prefix>&mode=count|hybrid	Top-10 prefix matches. `count` = all-time, `hybrid` = recency-aware.
POST	/api/search	Body `{"query":"..."}`. Buffers the count; returns `{"message":"Searched"}`.
GET	/api/trending?n=10	Trending queries by decayed recent score.
GET	/api/metrics	Hit rate, DB read/write counts, write-reduction factor, p50/p95/p99.
GET	/api/cache/debug?prefix=<p>	Which cache node owns a prefix, and current HIT/MISS.
GET	/api/cache/ring?sample=N	Key distribution across the cache nodes.

Examples

```
$ curl '/api/suggest?q=ip&mode=hybrid'
{"prefix":"ip","mode":"hybrid","source":"cache","node":"localhost:6390",
 "suggestions":[{"query":"iphone pro max","count":85470}, ...]}

$ curl -X POST /api/search -H 'Content-Type: application/json' -d '{"query":"best
python tutorial"}'
{"message":"Searched"}

$ curl '/api/cache/debug?prefix=how'
{"key":"how","owner_node":"localhost:6390","total_vnodes":450,
 "redis_key":"sugg:hybrid:how","hit_or_miss":"HIT"}

$ curl '/api/cache/ring?sample=5000'
{"nodes":["localhost:6390","localhost:6391","localhost:6392"],
 "vnodes_per_node":150,"distribution":{"...6390":1544,"...6391":1731,"...6392":1725}}
```

> **Response field `source`** on `/api/suggest` is `cache` (Redis HIT), `trie` (miss served in-memory), or `empty` — surfaced in the UI as the HIT/MISS routing badge with the owning node.

04 Design choices & trade-offs

Why cache (sizing)

For ~10M DAU × ~4 searches/day ≈ 460 writes/s and ~5 reads/search ≈ 2,300 reads/s average (~7k/s peak). Query popularity is Zipfian, so a small hot set serves most traffic — caching it keeps reads off the DB. Without a cache, every keystroke is a `LIKE 'pre%'` scan + sort on Postgres and p95 collapses under load.

Cache: global & distributed

- **Global**, not per-instance — one copy, one place to expire; avoids cross-instance coherence traffic.
- **Distributed across 3 Redis nodes** for fault tolerance (losing one node drops 1/3 of the cache, not all of it) — a resilience decision, not a throughput one.

Routing: consistent hashing

Each node owns a different key slice, so routing must be deterministic per key. Round-robin loses *where* a key lives; `hash % N` remaps almost everything when N changes. A hash ring with 150 virtual nodes per node remaps only ~K/N keys on membership change — more code than `%N`, but stable.

Freshness & eviction

- **Jittered TTL** (~45s suggest, ~8s trending, ±20%) is the primary invalidation mechanism — bounded staleness, zero bookkeeping, anti-stampede.
- **Write-around**: searches update the store, not the cache; the cache refills lazily on the next miss (write-through would recompute a top-10 on every write — wasted work).
- **LRU eviction** keeps the constantly re-hit hot prefixes; LFU would need an aging factor.

Writes: write-back batching

An in-memory buffer aggregates duplicate queries and flushes on size or interval as one additive UPSERT (`count = count + EXCLUDED.count`, so concurrent flushes add rather than clobber). **Trade-off:** a crash loses at most one un-flushed window — acceptable for approximate, self-healing counts. Durability would need a WAL or a durable queue.

Store: PostgreSQL

A write-heavy profile usually argues for an LSM/NoSQL store, but batching already cut DB writes ~4× in rows and ~1,000× in transactions, removing that pressure. So Postgres fits: B-tree index for prefix range scans, ACID for the count of record, read replicas for the rare miss-path read.

Serving & ranking

- **Trie with lazy top-k**: precompute candidate pools only for short prefixes (≤3 chars); compute longer ones on demand. Counts/recency read live, so one structure serves both modes.
- **Ranking**: `count` (all-time) vs `hybrid` = `w_pop·log(count) + w_rec·recent_score`, where recent_score decays exponentially (1h half-life) so spikes fade instead of ranking forever.

Consistency & the Next.js adaptation

Eventual consistency, **PA/EL**: serve stale rather than error during a partition (AP) and serve from cache rather than re-check the DB normally (EL). The one framework-shaped decision is the **runtime singleton**: Next.js has no single startup hook like Express's `app.listen`, so the trie, buffer, and Redis pool are built once and cached on `globalThis`, with concurrent first-callers awaiting one in-flight bootstrap promise.

Topic	Decision	Rejected alternative
Caching	cache reads	no cache → DB-bound p95
Locality	global	per-instance → duplication + coherence
Topology	distributed (3)	single → SPOF / stampede
Routing	consistent hashing	round-robin, `%N` (mass remap)
Eviction	LRU	LFU (needs aging)
Freshness	write-around + jittered TTL	write-through, targeted invalidation
Writes	write-back batching	sync per-write (DB-bound)
Store	PostgreSQL	NoSQL/LSM (unneeded after batching)
Consistency	eventual / PA-EL	strong (latency cost)
Serving	trie + lazy top-k	full precompute, DB-only
Ranking	count + decayed hybrid	raw recent counter
Lifecycle	`globalThis` runtime singleton	per-request init (reconnect storm)

05 Performance report

Setup: 1 Next.js server (`next start -p 8000`), 1 Postgres, 3 Redis nodes (Docker), all on one machine. Dataset: 120,000 synthetic Zipf-distributed queries. Reproduce with `npm run bench -- --reads 4000 --writes 6000`.

CACHE HIT RATE	SERVER P95	WRITE REDUCTION	TRIE ENTRIES
81.4%	**5.1ms**	**4.1×**	**121k**

Read latency — GET /api/suggest

- Server-side p95 ≈ **5 ms** (cache hit → Redis; miss → in-memory trie).
- Client-side p50 / p95 ≈ **5 ms / 12 ms** under 20 concurrent readers (includes HTTP + Next.js routing overhead).
- DB reads on the suggestion path = **0** — a miss is served by the trie, never Postgres.

> The server p95 (~5 ms) is higher than a bare Express handler (~0.5 ms); that gap is Next.js Route Handler / framework overhead, paid per request — a deliberate trade for the App-Router developer model.

Cache hit rate

~**81%** over 4,000 mixed reads (3,255 hits / 745 misses) on a synthetic Zipf prefix mix; climbs toward 90%+ with more skewed real traffic or a longer `TTL_SUGGEST`.

Write reduction — batching

Searches received	Rows written	Flush transactions
6,000	~1,460	4

≈ **4.1× fewer rows** (duplicate aggregation) and ~**1,500× fewer transactions** (batching).

Consistent hashing — distribution

5,000 sample prefixes across 3 nodes (150 vnodes each): **1,544 / 1,731 / 1,725** ≈ 30.9% / 34.6% / 34.5% — within ~±4% of even. Adding or removing a node remaps only ~1/N of keys.